



Efficient data race detection for interrupt-driven programs via path feasibility analysis

Jingwen Zhao¹ · Yanxia Wu¹ · Jibin Dong¹

Accepted: 3 May 2024 / Published online: 14 June 2024

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

Abstract

Interrupt-driven programs are widely used in embedded systems with high security requirements. However, uncertain interleaving execution of tasks and interrupts may cause concurrency bugs, with data races being a significant factor in threatening security. Most of the previous research has focused on detecting data races in multi-threaded programs. And existing static analysis methods for interrupt-related data race detection often produce many false positives. This paper presents IntRace, an accurate and efficient static detection technique for interrupt data race. IntRace eliminates false data race by analyzing potential concurrency relationships and path reachability. It first identifies all race candidate pairs using access interleaving pattern matching. Then for each pair of operational accesses, IntRace analyzes potential concurrency relationships, including the special case of interrupt nesting, and uses this information to filter out access pairs that cannot concurrently access the same location. Finally, it checks the feasibility of events in the access pairs by constructing path constraints, which effectively eliminating infeasible paths in concurrent contexts. In addition, IntRace was evaluated on benchmark tests and 9 real embedded programs. The experimental results show that IntRace reduces the false positive rate by 73.2% compared to recent studies.

Keywords Interrupt-driven programs · Date race · Static analysis · Path feasibility analysis · Bug detection

✉ Yanxia Wu
wuyanxia@hrbeu.edu.cn

Jingwen Zhao
zhaojw123@hrbeu.edu.cn

Jibin Dong
jibindong@hotmail.com

¹ College of Computer Science and Technology, Harbin Engineering University, Harbin 150001, China

1 Introduction

Interrupt-driven programs are widely used in embedded systems such as aerospace, defense and military, medical devices, mobile systems, and so on. These fields often have high security requirements, typically utilizing interrupts to provide concurrency and achieve interaction with hardware and timely response to peripheral devices [1]. However, since the execution of interrupt processes interacts is complex and unpredictable, the behavior of interrupt-driven programs can be uncertain. In some cases, programmers deliberately leave shared resources unprotected to achieve high real-time performance. If access to shared data is not synchronized or protected, uncertain interleaved execution of interrupts may lead to concurrent bugs [2]. Such as data race and atomicity violations caused by interactions between tasks and interrupt service routines (ISRs). This can result in serious security problems that are difficult to detect, locate, and reproduce. High property damage and loss of life [3–5] have made software developers and testers increasingly aware of the importance of defect detection.

Among the series of concurrent bugs, data race problems are most prominently caused by interrupts. Embedded software is usually designed with a large number of shared resources to implement data communication, where access to these shared resources is relatively frequent. The interaction between the main task and ISR is difficult to grasp in the design and implementation, which leads to the problem of data race from time to time. There is no absolute correspondence between the execution result of the data race problem and its existence of such issues, and interrupts only occur in specific states [6, 7]. As a result, locating and reproducing the cause of a failure can be a huge challenge.

In general, there are two categories of techniques for detecting data races in interrupt-driven programs. One approach [8–11] is to convert the interrupt-driven program into a multi-threaded program and use thread-level methods for detection. However, interrupts usually have different priorities, occur only when the hardware component is in a specific state, and may run in interrupt nesting which is different from detecting thread-level concurrency defects. The second category is direct analysis of interrupt-driven programs, which can be achieved through static and dynamic analysis. Dynamic analysis methods [7, 12–14] rely on executing the program to accurately gather information about its behavior. However, due to the need to establish a target execution environment, these methods have high scheduling overhead and are difficult to apply to large-scale real embedded software.

Numerous tools use static analysis methods [15–21] for interrupt data race detection, which can process large-scale codes and perform in-depth analyzes without running the program. To address the high false positive rate of static analysis, several researches have been conducted to improve the accuracy of the analysis. Wang et al. [16] proposed the method of combining static analysis and dynamic simulation. Feng et al. [17] implemented interrupt analysis using CBMC and proposed IML to describe the synchronization of interrupt driver programs. However, they did not consider the implicit dependencies between tasks and ISRs, lacked information about program semantics, and could not handle

interrupt nesting well. Chopra et al. [19] introduced the concept of precedence order and proposed disjoint blocks to define synchronization. However, interrupt-driven programs often use self-organizing synchronization mechanisms, which can lead to false positives. Therefore, it is important to design a targeted analysis method based on interrupt characteristics and develop an accurate detection tool to effectively eliminate false positives from static analysis.

In this paper, we propose IntRace, a tool that combines static analysis and constraint solving to accurately and effectively detect data race defects in interrupt-driven programs. Our method can be divided into three stages: interleave pattern matching, potential concurrency detection, and path feasibility analysis. We eliminate false positives gradually through more accurate analysis at each stage.

In the interleaving pattern matching stage, IntRace performs an inter-procedure data flow analysis for every task or interruption to extract all necessary information for defect analysis. Meanwhile, the tool ignores synchronization conditions, matches access interleaving with the proposed access interleaving patterns to identify all the potential data race candidate pairs. It's necessary to validate each matched race candidate pair after static analysis for eliminating false alerts. In the second stage, we first analyze the function statements associated with the events in each candidate pair in isolation which is divided into two basic blocks, then analysis by synchronization conditions, priorities, and timing. Then the tool verifies pairs according to if a potential concurrency relationship exists between the two blocks. Furthermore, we delete candidate pairs that are unlikely to be executed concurrently, thus reducing unnecessary costs in the third stage. In the feasibility analysis phase, we construct the symbol summary of each interrupt function and insert the high-priority blocks corresponding to the candidate pairs into the low-priority blocks at the preempted positions. IntRace constructs constraint conditions and uses constraint rules to convert the semantics and attributes to be validated into constraint expressions. We use the SMT solver to verify the reachability of each parallel path for obtaining real data race.

To verify the effectiveness of IntRace, we evaluated it on the Racebench [22] benchmark and 9 real programs. The experimental results on Racebench show that the method can analyze interrupt behavior more efficiently and accurately. IntRace can detect all data races in Racebench and significantly reduce the number of false positives compared with the existing method. The results from the 9 real world programs show that this method can be applied to larger-scale interrupt-driven programs and can obtain higher precision analysis results. Also, for a real program consisting of several thousand lines, the analysis typically takes only tens of seconds.

In summary, the contributions of the paper are listed as follows:

1. Proposes a new static data race detection method for use in larger-scale interrupt drivers.
2. Supports analysis of potential concurrency relationships and semantics of interrupts, effectively and accurately eliminate infeasible paths.
3. Evaluates results on both benchmark and real-world programs to demonstrate the effectiveness of this method.

The rest of this paper is organized as follows. Section 2 introduce background and a motivating example. Section 3 give a detailed description of IntRace. We present our experimental evaluation in Sect. 4. We review the related work in Sect. 5. Finally, Sect. 6 concludes the work.

2 Background and motivation

In this section, we first give the background, then use an example to illustrate the challenges of data race detection in interrupt-driven programs.

2.1 Interrupt-driven program

Interrupt-driven programs are common design pattern in embedded software. In general, an interrupt-driven program consists of a main program that includes one or more tasks and a plurality of ISRs. Typically, when a processor receives an interrupt signal, it halts the current activity being executed, saves the current active state, and executes an ISR in response to the event. After the ISR finishes, the processor resumes normal activities. We can represent an interrupt-driven program by tuple (P, V) , where P represents one or more tasks and a set of ISRs, and V represents a set of global variables. It need to be mentioned that each ISR has its own priority, and read and write events occur on shared variables.

Interrupt-driven programs differ from general multi-threaded programs in their scheduling mechanism, synchronization mechanism and preemption relation [8]:

- *Scheduling mechanism* Interrupts occur randomly and unpredictably. The processor switches the ISR at different times according to various needs, scheduling it using information such as priority. Once an interrupt starts, it will continue to execute until it ends, unless a higher priority interrupt preempts it. Because of this, the internal state of the interrupt is invisible to tasks and other interrupts.
- *Concurrent primitive mechanism* Interrupt-driven programs do not have a specific concurrency primitive mechanism, so a self-organizing synchronization mechanism is often used. A typical way to implement concurrency control is through disabled-enable interrupts and flag-based synchronization. Interrupts are highly dependent on hardware state and occur when the hardware is in a specific state [12], and no other operation can be inserted in between.
- *Asymmetric preemption relationships* In an interrupt-driven program, there is an asymmetry between normal tasks and interrupts, as well as between different interrupts. When interrupts with priority are set to preemptible, interrupts can preempt normal tasks [6]. If no higher priority interrupt arrives, the execution of the current interrupt will run to completion. When multiple interrupts are preempted simultaneously, interrupt nesting occurs, which can make program behavior very complex.

2.2 Date race

When two different interrupts (or tasks) simultaneously access the same shared data for read and write operations, and at least one of them is a write operation, a data race occurs [23]. Because the timing of the two accesses is not determinable, the program may produce abnormal behavior. For example, since critical data may be unexpectedly modified, inconsistent or erroneous read values, and even system or software crashes may occur in severe cases.

We provide a formal description of data race in interrupt-driven programs. Specifically, when a task or ISR accesses memory where a shared variable is located, another high priority ISR preempts it and performs an operation on the same shared variable.

$$\begin{aligned} \forall event_1, e_2 \wedge e_1 \in T_1, e_2 \in T_2, T_1 \cap T_2 = \emptyset \wedge W(e_1) = W(e_2) \wedge W(e_1) = W(e_2) \\ \wedge obj(e_1) = obj(e_2) \wedge p(e_1) < p(e_2) \wedge (op(e_1) = Write \vee op(e_2) = Write) \end{aligned} \quad (1)$$

Each access event e_i consists of a five-tuple $e_i = (T_i, W_i, obj_i, op_i, p_i)$, where T_i is a task or ISR, W_i is the code address of the access event. p_i is the priority of the interrupt, with larger number denoting lower priorities. And $op_i(Write, Read)$ represents an access operation to the same shared variable obj_i . e_p, e_c are two access events operating on the same shared variable in one execution, where e_p come from a task or the interrupt ISR_1 with lower priority, while e_c comes from different higher priority interrupt ISR_2.

In general, data race errors caused by incorrect access interleaving of shared variables can be induced by interrupts [24]. While this type of error has been found in real projects, the characteristics of data races can be described using error patterns, as shown in Table 1.

Inspired by existing research, we examine task/low-priority interrupt events e_p and high-priority interrupt events e_c , to determine whether a feasible path exists between successive instruction regions in the program where these two events occur. In other words, for each access event pair (e_p, e_c), if the sequential instruction regions in which the two events occur are interleaved during program execution, and if the corresponding order is executed according to the asymmetric preemption relationship. For this, we say that the two events participate in data race.

Table 1 Error mode description

No	Bug patterns	Description
1	$W_1 - W_2$	The task write is overwritten by the interrupt write
2	$W_1 - R_2$	The task write is read by the interrupt, the programmer may want to interrupt read other values
3	$R_1 - W_2$	The task read is modified by interrupted write

2.3 Motivative example

The detection of interrupt data races mainly faces the following three challenges:

The first challenge of embedded software is that it uses special operations to control interrupts and priorities to affect the order relations between concurrent events. As shown in Fig. 1, `_disable_irq( )` and `_enable_irq( )` are used to disable and enable interrupts. If an interrupt is disabled, failure to identify such operations would result in false positives. Additionally, shared variables in ISR1 cannot be preempted because ISR2 has lower priority. For example, the content of `currentInput` at `line_22` cannot be modified by the write of `currentInput` at `line_28`. Therefore, it is impossible for these two events to occur between them. In summary, not all interrupts can be performed, and it is necessary to filter out pairs of access events that cannot access the same location from the results.

The second challenge is the complexity of program analysis due to interrupt nesting and loops, where ISRs have different priorities, as shown in Fig. 2. High priority interrupts can preempt low priority interrupts when the latter preempts the main task, forming a nested situation. At the same time, low priority interrupts may also affect high priority interrupts. To address this, the analysis can be constrained layer by layer based on the different priorities. Additionally, limiting the loop unwind depth can help mitigate this problem.

A third challenge is that false positives may be reported if data flow changes due to interrupts are not considered during program execution. In Fig. 3, the ISR does not satisfy the conditional statement (`fData = buffer[1]`). The access operation in the ISR part is not possible to execute in practice, which means it is not a data race. Therefore, it is necessary to verify whether these competing interweavings

Fig. 1 A data race case

```

1  int main(){
2      _enable_irq();
3      //Open interrupt, close the interruption for _disable_irq( )
4      ...
5      input = *(IN);
6
7      S1: if(fg == 0xFF){
8          currentInput = 2 * input * PI;
9      }
10     ...
11     S2: if(fg == 0xFF){
12         *OUTPUT = currentInput;
13     }
14 }
15
16 void ISR1(){
17     ...
18     S3: if(fg == 0xFF){
19         currentInput = input + 1;
20         fg = 0xFE;
21     }
22     c = currentInput;
23     ...
24 }
25 void ISR2(){
26     ...
27     S4: if(fg == 0xFF){
28         currentInput = 0.75 * input * PI;
29     }
30     ...
31 }
```

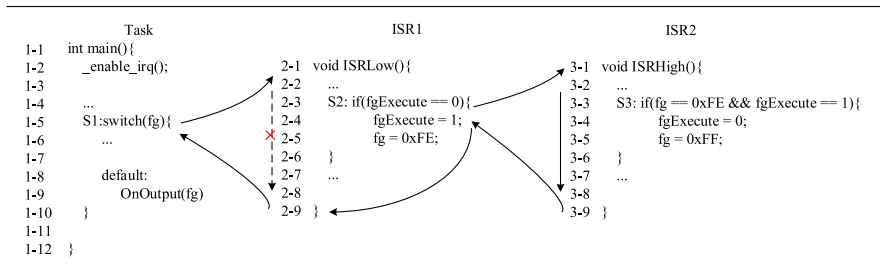


Fig. 2 Interrupt nesting situation

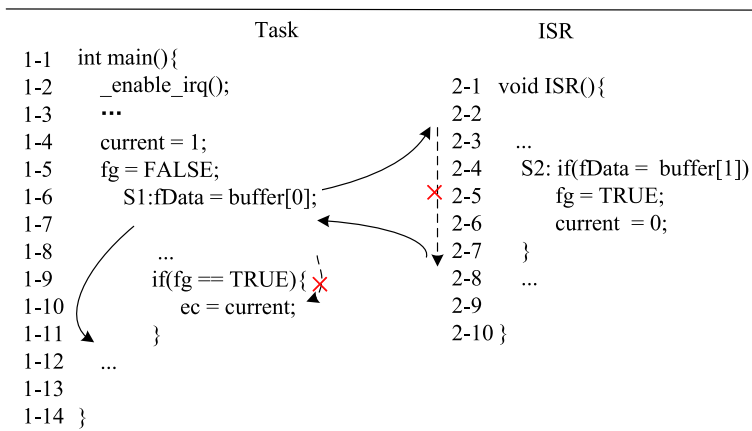


Fig. 3 A case of infeasible interleaving

are feasible by analyzing interweaving semantics and solving path constraints. This involves checking whether the path between two events in an access pair is reachable. False positives caused by infeasible access interleaving can be effectively eliminated.

3 Approach

The main idea of IntRace is that more accurate analysis is gradually introduced to eliminate false positives. The IntRace framework, shown in Fig. 4, it mainly includes three stages: interleaving pattern matching (Sect. 3.1), potential concurrency relationship analyzing (Sect. 3.2) and access pairs feasibility checking (Sect. 3.3).

In the first stage, we match all pairs of suspicious data race access events using three interleaving patterns. To do this, we first use Clang's abstract syntax tree and static parser capabilities, LLVM-PASS, to analyze the LLVM-IR code of the source program to build a control flow graph. Based on the information extracted for data race analysis from the tree structure, such as read and write operations on shared

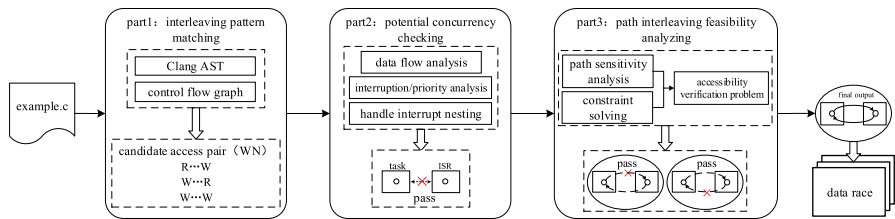


Fig. 4 IntRace framework. Includes three stages: interleaving pattern matching, potential concurrency relationship analyzing and access pairs feasibility checking

variables, we ignore the statement guard condition. However, many false positives are reported in this stage, by identifying race candidate pairs by matching data race interleaving patterns.

In the second stage, the tool treat the task or interrupt where the two events are located as region blocks. We identify the synchronization operations related to interrupts, add variables to record the priority and interrupt period, and check the potential concurrency relationship between the two region blocks corresponding to the candidate pairs by data flow analysis. For each pair of competing candidate events, we analyze the execution status, calling time, priority and other information of the interrupt, and add user profiles to specify the relevant implicit interrupt API names. For the special case of interrupt nesting, we use a step-by-step sequential conversion from inside-out to handle it, which filter out access event pairs that cannot access the same location concurrently.

In the third stage, we perform a path-sensitive analysis of execution events within two regional blocks. The tool check whether there are execution paths between the remaining competing candidate pairs that can trigger defects. We transform the problem into an event reachability verification problem for serialized programs, specifying the cycle development depth. Constraints are constructed, and constraint solving using z3 is performed to eliminate infeasible interleavings.

The specific methods of each stage will be described in the following subsections.

3.1 Interleaving pattern matching

In general, data races are mainly caused by inappropriate access to shared variables by two operations. Therefore, how to accurately extracting relevant information is an important part of detection. We extract the necessary information for defect analysis by using Clang's Abstract Syntax Tree (AST) and static analyzer checker to gather program execution information. Programs are divided from coarse to fine: files, functions, code blocks, and expressions.

For one thing, we need to identify which variable accesses are related to possible shared variables. Because some kinds of variables cannot be shared by multiple threads, such as local variables and non-pointertyped function arguments, and thus variable accesses are not useful to data race detection. We identify variable accesses affected by global variables and pointer-type function arguments, when performing static analysis, which are often the source of shared variables [25]. For each detected

shared resource, we also add all its aliases to the shared resource set. We perform inter-procedural alias analysis of the program from top to bottom based on the call graph. For each function, when a variable is used as an argument of the called party, we record its alias in the caller, aliases of the formal references in the called party, and their relationship.

For another, two levels of information are obtained in the overall phase of extraction. The first level describes the structure of the code, including the calling relationship between functions, code location, interrupt function entry and exit points, all shared variable operations, instructions and functions corresponding to shared variables, etc. The second is the extracted information of the control flow chart of each task, which is mainly about the distance relationship between concurrent operations and the characteristics of the operations themselves, whether it is inside the function body. We also consider the hardware components that the program can access, including device ports and registers. We establish a shared resource pool P to identify variables that are commonly accessed by at least two different ISRs or a task and an ISR. For each shared variable detected, we add it to the library.

Algorithm 1 Match potential data race access pairs

Input: P

Output: Potential data races C

```

1:  $C \leftarrow \emptyset$ ;
2: for  $e_p \in T_p, e_c \in T_c$  in  $T$  do
3:   for each  $obj(e_p) \in E_p$  do
4:     for each  $obj(e_c) \in E_c$  do
5:       if  $p(e_p) < p(e_c)$  then
6:         if  $obj(e_p) = obj(e_c)$  and  $(op(e_p) == Write \vee op(e_c) == Write)$  then
7:            $output(C)$ ;
8:            $C \leftarrow C \cup \langle e_p, e_c \rangle$ ;
9:         end if
10:      end if
11:    end for
12:  end for
13: end for

```

In order to avoid missing any defects, we ignore interrupt states and statement guard conditions (analyzed in Sect. 3.2) at this stage and match race candidate pairs that may compete with each other from the shared repository. Algorithm 1 shows the matching process of race candidate pairs. In this algorithm, we extract all shared variable read and write accesses from the shared repository. Denote by E_i , the set include events on a shared variable $obj(e_i)$ of a task or interrupt t_i . For a shared variable $obj(e_i)$ in a Control Flow Graph (CFG), it is accessed by at least two events from different tasks or interrupts. We check whether the combination of the two events matches one of the access interleaving patterns. That is, at least one of the access operations is a write and has a different priority. Then, the pair of events

$\langle e_p, e_c \rangle$ forms a potential race condition and is added to the data race candidate pair set C .

3.2 Potential concurrency relationship analyzing

At this stage, we eliminate some subsets by analyzing the potential concurrency relationships between tasks/interrupts of the race candidate pair $\langle e_p, e_c \rangle$. As shown in Fig. 5, each task or interrupt forms a basic block consisting of a quadruple (T_i, I_i, S_i, p_i) , where T_i is the name of the task or interrupt, I_i is the interrupt status, S_i is the time or period when the interrupt occurs, p_i is the priority. In general, an unambiguous (or normal) basic block should fulfill the two requirements. First, the basic block should contain the start and end points of the task or interrupt. Second, it should contain the related and associated operations of shared variables. If there is no concurrency between the two basic blocks, or impossible interrupt timing results in no overlap in execution time, then no data race will occur. That portion of candidate pairs can be screened out. On this basis, in order to improve the accuracy of the analysis, we design the interrupt nesting processing mode. The specific method is as follows.

We identify synchronization operations related to interrupts, including implicit interrupts. In many embedded software systems, coding interrupt operations can be rather flexible, non-standard synchronization mechanisms are often used to control interrupts. These mechanisms include disabling/enabling interrupts, hanging the scheduler, and more. It is important to ensure that these mechanisms can be analyzed correctly. For standard interrupt APIs, including `disable_irq_all()`, `disable_irq(int n)`, `disable_irq_nosync(int n)` and `enable_irq(int n)`, where n indicates the unique ID of the interrupt. Such as `_disable_irq( )` and `_enable_irq( )` shown in Fig. 1, these operations can be identified directly through static analysis. For implicit operations, it's provide a configuration file that allows programmers to specify the name of the interrupt APIs, the code location, and the type of operation (enabled or disabled). Each data control flow is traversed using depth-first search to analyze the state corresponding of the

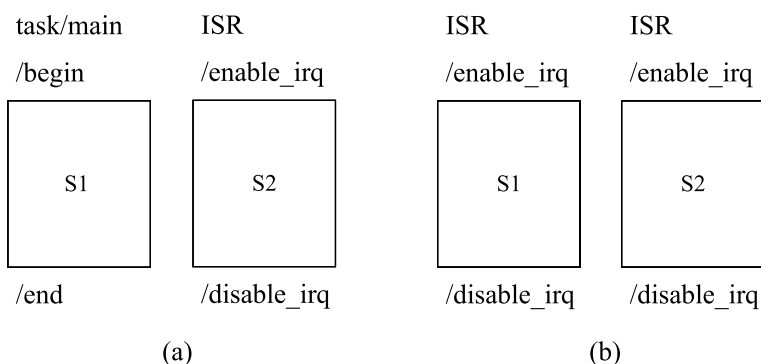


Fig. 5 Two common basic block forms

interrupt. The state is recorded as (1, 0), where 1 denotes that the interrupt is open and can be preempted, 0 denotes that the interrupt is off and preemption does not occur.

At the same time, we need take into account the implicit dependencies between tasks and interrupts due to priority. Because of the asymmetric preemption relationship of the interrupt program, task threads can be preempted by other task threads or interrupt programs (as long as interrupts are not disabled). Low priority interrupts may be preempted by high priority interrupts. We use the auxiliary variable `I_level` to record the interrupt priority. The higher the number, the lower the priority. In the example of Fig. 1, the `I_level` of `ISR_2` is 2 and the `I_level` of `ISR_1` is 1, indicating that `ISR_1` has a higher priority. Therefore, the shared variable contents in `ISR_1` cannot be modified by a write in `ISR_2`. We which indicate that the data race is due to access to the same shared variable in T_1 and T_2 . If T_2 has a lower priority than T_1 , then preemption does not exist and no race occur.

However, not all high priority preemption affect the results, as interruptions can occur in an uncertain manner at any time. For example, after the low priority interrupt event operation e_p is completed, the high priority interrupt may start after the low priority interrupt is finished or preempt. In such cases, there is no interleaving of the shared variable. Therefore, it is necessary to consider when preemption occurs.

Some interrupts have a periodic character and are triggered only at specific times or within specified periods, requiring analysis of timing relationships. At the same time, the execution path may differ depending on when the interrupt preemption occurs, even if the same input. Therefore, it is important to determine the arrival time of each interrupt. In each preempted task or interrupt, we add a helper variable `op_time` to record the point in time that the event started being preempted. The variable `op_time` is initialized to 0 and marked as 1 in the program when a high priority interrupt is triggered. By these methods, record the position in the data stream at this time, extract the interrupt cycle interval information, and collect the overlapping area of their execution time. The operation corresponding to a critical access in one interrupt preemption starts in the lifetime of the operation corresponding to another access.

For the access pair $\langle e_p, e_c \rangle$, the time conditions $Time(e_p, e_c)$ can be verified with data control flow. We construct time overlap conditions for T_1 and T_2 , where T_1 and T_2 both start when the task is executed or the interrupt is turned on, respectively, and end when e_p and e_c are completed. Given the start of a task or low priority interrupt, and the time position of the high priority preemption. The time condition of the basic block where the event is located can be calculated by determining whether the information state has an intersection.

$$Time(T_i, T_j) = ((TT(v_j, v_{j+1}) \wedge OP(T_j) \wedge CI(T_j)) \subseteq ET(v_j, v_{j+1})) \quad (2)$$

As shown in the above formula, the time condition includes following parts:

1. $TT(v_j, v_{j+1})$ denotes the information of the high priority interrupt from the start of the preemption flow until e_c is completed.

2. $ET(v_i, v_{i+1})$ denotes the execution information of the event e_p data stream in the preempted task or interrupt.
3. $OP(T_i)$ denotes the high priority interrupt is enabled.
4. $CI(T_j)$ denotes whether the value stream time of $TT(v_j, v_{j+1})$ is within the specified periodic interval.

Algorithm 2 Potential concurrent relationship analysis

Input: Potential data race $\langle e_p, e_c \rangle$

Output: More accurate potential data race $S = \langle e_p, e_c \rangle$

```

1:  $S \leftarrow \emptyset$ ;
2: for  $e_p \in T_1$  and  $e_c \in T_2$  do
3:   for  $T_1$  is a task or ISR, and  $T_2$  is an ISR do
4:     if  $I(T_1) = 1$  and  $I(T_2) = 1$  then
5:       if  $I\_level(T_1) > I\_level(T_2)$  then
6:          $dataFlowAnalysis(T_1, T_2)$ ;
7:         if  $isSatisfaction(Time(T_1, T_2))$  then
8:            $S \leftarrow S \cup \langle e_p, e_c \rangle$ ;
9:         else
10:          find another possible candidate;
11:        end if
12:      end if
13:    end if
14:  end for
15: end for

```

Algorithm 2 shows the process of potential concurrency relationship analysis. The task and interrupt, interrupt and interrupt are classified and analyzed. Firstly by examining the event e_p and e_c and determine the task / interrupt in which they are located, the synchronization and priority order are checked between them respectively. Then, the algorithm verify the timing relationship, based on the data flow collection overlapping execution time, and add the race candidate pair $\langle e_p, e_c \rangle$ to the more accurate data race candidate set S cording to the condition.

Interrupts can also be called in a nested fashion, as shown in Fig. 2. The behavior of one ISR may be affected by other ISRs, which complicates the information collected by static analysis process. When an interrupt function is called before another interrupt function returns. Meanwhile, a higher priority interrupt function is called before the called interrupt function returns, which there are nested interrupts. In general, the innermost interrupt has the highest priority level. Interrupt nesting can be thought of as a special kind of function call, but the difference between them is that the triggering of interrupts is non-deterministic and requires the introduction of priority in the data control flow between interrupts. Interrupt nesting analysis requires:

1. Separate analysis of each interrupt.
2. Sequential conversion from inside to outside, passing the shared variable operation to other threads.
3. Enough consideration of all possible interleavings.

Assuming that the entire program contains a finite number of interrupts, each interrupt routine consists of a control flow graph $CFG = (N, \zeta)$. $(n_0, n_1, \dots, n_n \in N)$ denotes each operation in the interrupt program, where n_0 is the entry operation for the low priority interrupt, ζ denotes the transition state of the current operation to the next operation, V is the set of operation nodes processed in the interrupt, OC is the set of unprocessed operation nodes in the interrupt, which are program statements that have not yet been traversed. This process continually culls processed operations from the OC until all operations have been traversed. Let the preemption D_j of ISR_1 be the interference of operation D_j . When a high priority interrupt interference is identified, it is inserted into the original operation to perform preemption. The function CDR (Constructed Dependency Relation) takes the memory state as input and returns a new memory state as output. Otherwise, if there is no preemption, the next operation of the original operation is traversed. Until all operation statements within that interrupt have been traversed, which means that the OC is empty.

Algorithm 3 Analysis of interrupt nested program

Input: Compare and select the higher priority interrupt ISR_1, ISR_2
Output: Different interleaves execute corresponding sequential programs

```

1:  $ISR\_2$  CFG  $(N, \zeta)$ 
2:  $V \leftarrow \emptyset$ ;
3:  $OC \leftarrow \{(n_0, n_1, \dots, n_n \in N)\}$ ;
4: while  $\exists n_i \in V$  do
5:   for  $analyze\_dependencies(n_i)$  do
6:     if  $shared\_variable(n_j) == 1$  then
7:        $identify(D_j)$ ;
8:        $v \leftarrow CDR(n_j, V \cap D_j)$ ;
9:     else
10:       $v \leftarrow CDR(n_{j+1}, V)$ ;
11:    end if
12:  end for
13:  until Set  $OC$  is all traversed
14: end while
15:  $V \leftarrow V \cap v$ ;
16:  $OC \leftarrow \emptyset$ ;
17: return
```

Algorithm 3 shows a specific operation, that involves analyzing three interrupts ISR_1 , ISR_2 and ISR_3 , $p(ISR_1) > p(ISR_2) > p(ISR_3)$. The algorithm first selects the higher priority ISR_1 and ISR_2 for analysis, and precisely

checks all possible behaviors of the two internal interrupts. It begins by analyzing the full execution semantics of each ISR individually, capturing the preemption of ISR_2 by the high priority interrupt ISR_1 , and calculating the interference. Since preemption at different operation nodes of low priority interrupt ISR_2 can lead to different program behavior. The algorithm analyzes the data dependencies between operational nodes and constructs a data dependency graph [26], that analyzes each interrupt again on this basis. For the preemption or operation dependency of high priority ISR_1 , the algorithm insert them in the correct position of ISR_2 in turn. While the process is iterated continuously, the interrupts is converted into a serial program according to different execution interleavings. Finally, the sequential program consisting of two interrupts and the lowest priority interrupt is analyzed by Algorithm 3, which is put into an analysis of two interrupts with different priorities.

3.3 Access pairs feasibility checking

Among race candidate pairs, some behaviors never occur in actual execution. Considering the phenomenon in Fig. 3 when the ISR preempts the task, the event cannot occur because the task's state condition ($fData = buffer[0]$) does not satisfy the path condition ($fData = buffer[1]$) for the ISR. Therefore, in order to verify whether race candidate pair is true, a feasible path needs to be found between events, to avoid unexpected false positives. In this section, we transform this problem into a reachability verification problem. Since data races usually occur in a small subset of program statements, we consider only consider the relevant statements. It's check the interleaving feasibility between candidate pairs $\langle e_p, e_c \rangle$, the preemption position Q of the interrupt pair e_p where the mark e_c is locate, and the conditional statement in the path. We observe whether the sub-paths are all feasible and there are operations occurred after the interrupt returns. Meanwhile, we construct the symbol summary of each interrupt function to avoid redundant path condition computation. For each race candidate pair, IntRace selects representative preemption points to reduce the interleaving space.

First, we need to obtain the path information and correctly insert the high priority interrupt into the low priority task or interrupt. Existing methods do not take into account the uncertainty of interrupt interleaved execution during collecting path conditions. It's inevitable for traversing all program statements directly that state space explosion will happen, which exceed the solving ability of the constraint solver.

Considering a data race candidate pair $\langle e_p, e_c \rangle$ where e_c is in an interrupt with higher priority than e_p is in a task or interrupt, we add an auxiliary variable I_op to the high priority interrupt to check whether it is preempted. When $I_op=1$, it means that e_c has been traversed and executed, preempting the low priority task or interrupt. At this point, we insert the assert property verification statement into the program, and use the constraint solver to verify whether we can reach the read/

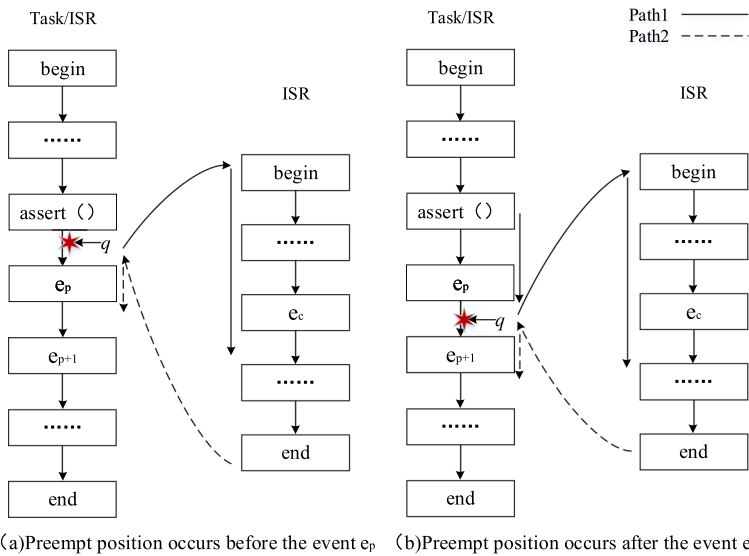


Fig. 6 Two types of preemptive situations

write locations in the high-priority interrupts associated with potential contention problems, and analyze the feasibility between ISR_{open} and e_c . Simultaneously, the path is captured according to the preemption position. As shown in Fig. 6, if the preemption position occurs before the event e_p , the path condition from the preemption position q through the interrupt open point q through the interrupt open point ISR_{open} to the return point ISR_{end} of the interrupt where e_c is located needs to be obtained. And a path condition from the interrupt return point ISR_{end} of e_c to e_p via the preemption position q should be obtained, which means verifying whether both paths $(q \rightarrow (ISR_{open} \rightarrow \dots \rightarrow e_c \rightarrow \dots \rightarrow ISR_{end}))$ and $(ISR_{end} \rightarrow q \rightarrow e_p)$ are feasible. Similarly, if the preemption occurs after the event e_p , it is necessary to obtain whether the paths $(e_p \rightarrow q \rightarrow (ISR_{open} \rightarrow \dots \rightarrow e_c \rightarrow \dots \rightarrow ISR_{end}))$ and $(ISR_{end} \rightarrow q \rightarrow e_{p+1})$ are both feasible. In this way, we construct an execution path containing candidate pairs, which path consists of events associated with the read and write variables.

Then, we construct constraint conditions and use constraint rules to convert the semantics and attributes to be validated into constraint expressions. And it's necessary to check whether two paths are feasible. If a path is infeasible, we prune it and delete the candidate pair. If both paths are feasible, we report the data race error to the user.

Algorithm 4 Path condition construction

Input: Potential data race $S = \langle e_p, e_c \rangle, ISR$

Output: Real data race R

```

1:  $R \leftarrow \emptyset$ ;
2: for each  $Hpriority\_ISR(e_c)$  do
3:   if  $ISR$  meet  $assert()$  condition then
4:      $Q \leftarrow preemptPosition(e_q)$ ;
5:   else
6:     delete  $\langle e_p, e_c \rangle$ ;
7:   end if
8: end for
9: for each  $q$  from  $Q$  do
10:   $PC(P_1) = constructPC(ISR_{open}, e_c)$  ;
11:   $PC(P_2) = constructPC(ISR_{end}, e_p)$  ;
12:  if  $Satisfiable(PC(P_1), PC(P_2))$  then
13:     $R \leftarrow R \cup \langle e_p, e_c \rangle$ ;
14:     $reportRace(R)$ ;
15:  else
16:    delete  $\langle e_p, e_c \rangle$ ;
17:  end if
18: end for

```

Algorithm 4 shows the main idea of constructing path conditions [27]. First we analyze two interrupts separately to build their summaries, which can be reused directly during interleaving feasibility checking. IntRace determine if $(ISR_{open} \rightarrow \dots \rightarrow e_c \rightarrow \dots \rightarrow ISR_{end})$ in the high priority interrupt satisfies the assert property verification statement (Line 3–6) and check the global state at the end of preemption. As shown in Fig. 3, we check whether the conditional statement $*lif(fData=buffer[1])$ in the ISR can be satisfied based on the statement $fData=buffer[0]$ in the task. If not satisfied, we eliminate the relevant paths directly. Otherwise, we construct the path conditions of two paths separately and use the SMT solver to verify whether both paths are reachable.

For the path $(q \rightarrow (ISR_{open} \rightarrow \dots \rightarrow e_c \rightarrow \dots \rightarrow ISR_{end}))$, we analyze the feasibility of the path between ISR_{open} and e_c . Path condition PC can be constructed by exploring control flow graph and data dependence graph. The core idea is to examine the sum taking condition of the two paths $Path_1$ and $Path_2$. Thus, the path condition $PC(ISR_{open}, e_c)$ can be constructed by connectivity conditions of the paths $Path_1$ and $Path_2$, where $Path_1$ and $Path_2$ each starts from an entry point (preempting the location q) and finishes in ISR_{open} and e_c . Thus, given a path $Path_1$ starting from an entry point, Var_i is the event variable on which the path condition depends, the path condition for $Path_1$ consists of 3 parts:

1. The conditions of the control flow graph for access to ISR_{open} from the preemption point q .
2. The feasible data flow between Var_i and Var_{i-1} .

3. The value flow from Var_i to Var_{i-1} is determined to be feasible or infeasible.

The path condition of $Path_2$ (from entry point to e_c) can also be obtained from these parts. Therefore, the path condition of path $(q \rightarrow (ISR_{open} \rightarrow \dots \rightarrow e_c \rightarrow \dots \rightarrow ISR_{end}))$ is:

$$PC(ISR_{open}, e_c) = PC(Path_1) \wedge PC(Path_2) \quad (3)$$

Similarly, we can obtain the path condition of path $(ISR_{end} \rightarrow q \rightarrow e_p)$ (from the global state at the end of high-priority interrupts to the return location of low-priority interrupts). Figure 7 describes the process of constructing path conditions, which we demonstrate using the example in Fig. 3. If both path conditions of the two paths are satisfied, it means that a real data race bug.

For loop statements, we specify the loop unrolling to a limited depth. The expansion is performed according to the expansion depth, and the problem is converted into whether the value of the constraint expression is true or not. When testing the feasibility of a path, if the path exploration cannot reach the state of Var_i , we check whether Var_i is in a loop. In such a case, we increase the number of loop iterations to the specified unwrap depth and try again to increase the chance of detection.

4 Experiment and evaluation

We have implemented IntRace, an accurate static detection tool for checking data race in interrupt-driven programs. It builds on a variety of open-source tools, such as Clang /LLVM [28] for implementing the C front-end, IntAbs [15] for implementing the interrupt nesting interleaving semantic analysis, and Z3 [29] for solving the path constraints. We conducted a comparative experiment between IntRace and the recent related work Rchecker. We carried out our experiments on a computer with

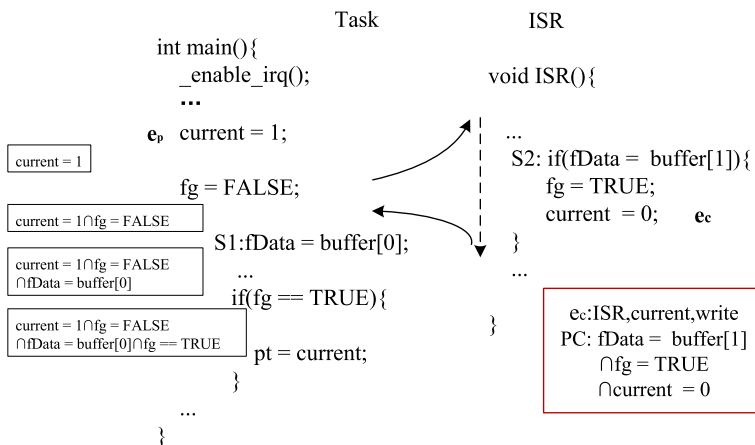


Fig. 7 The process of constructing path conditions

an Intel Xeon E5-2630 CPU, 32 GB of RAM, and the Ubuntu 16.01 Linux operating system.

To evaluate IntRace we consider three research questions:

RQ1: How effective is IntRace at detecting data race for interrupt-driven programs?

RQ2: How effective is IntRace at each stage of analysis?

RQ3: How efficient is IntRace at detecting data race for interrupt-driven programs?

4.1 Objects of analysis

Rchecker [17] is a data race detection tool for interrupt-driven programs based on CBMC. It has been evaluated on Racebench and we compared it with IntRace using the same benchmark. Unfortunately, the Rchecker implementation is not publicly available and has only been evaluated on Racebench. We were not able to obtain access to it during the early development of IntRace. Other available defect detection tools are mostly applicable to multi-threaded programs, and thus incompatible with the benchmark programs in our evaluation. We attempted to implement Rchecker's approach to defect detection, namely (1) using CBMC to extract all shared read/write accesses to construct shared access spaces, and (2) using the Interrupt Mask List (IML) to process synchronization. However, to ensure the accuracy of the data, we still used the data in Rchecker directly. In order to ensure the comprehensiveness of our evaluation, we selected 9 real industrial cases for our experiments. The two data sets used in our experiment are described as follows:

The interrupted data access conflict benchmark set Racebench consists of 31 cases with 50 manually inserted data race bugs. Many studies on interrupt driven program detection have performed their experiments on this suite. These cases are based on real aerospace embedded software characteristics, which can reflect the semantic characteristics of real-world interrupt concurrent programs. They cover six categories of key elements that affect data race detection and reflect most of the syntactic semantics and complex data types of interrupt concurrent programs. Although the case lines are relatively short, each case covers one or more data race issues, with hundreds or even thousands of access patterns. Besides, Racebench considers the different scenarios where the bugs happen, which helps evaluate the approaches comprehensively and makes the analysis challenging.

The 9 real industrial cases come from open-source projects and real-world test libraries of embedded software. First, we selected a device driver `mv643xx_eth.c`, and two driver programs from LDD [30]: `short` and `shortprint`. Three subjects (No.4–No.6) are real embedded software from the China Academy of Space Technology. The bugs in this database were reported by an independent validation and verification facility. Data race bugs have been detected in multiple software projects over the past few years. We selected representative cases based on selection criteria designed to cover multiple circumstances of the above properties. And all other interrupt programs are provided by SUNG in [15], which are open source and available online. Table 2 lists all programs, the number of lines of non-comment

Table 2 9 real industrial cases

No.	Name	Size (Loc)	#ISR	Description
1	mv643xx_eth.c	3256	1	A device driver mv643xx_eth.c
2	Short	704	5	A LDD driver program
3	Shortprint	531	1	A LDD driver program
4	Example1	1710	2	A thermal control software
5	Example2	2273	3	A power supply control software
6	Example3	3765	2	A power control software
7	i2c_pca_isa-3	364	8	A linux kernel driver for ISA boards
8	wdt_pci_1	1240	4	An industrial computer driver program
9	i8xx-tco-1	758	3	A TCO timer driver program

code they contain, the number of interrupts (with different priorities), and code description, each case covers hundreds to thousands of lines of code.

4.2 RQ1: effectiveness of IntRace

Table 3 shows the test results of Rchecker and IntRace on the Racebench benchmark set. Rchecker is not available and has only been evaluated on Racebench. One case in Racebench could not be analyzed by Rchecker due to an unexpected error. To ensure comparison, we also excluded this case in our experiments. #ISR is the number of interruptions, #Race is the number of data races in the benchmark test. Columns 5–8 show the results of the Rchecker tool, including the number of errors detected, real errors, false positives, and detection times. Columns 9–11 of the table show the results of IntRace detection, #Num is the number of data races detected, #FP is the number of false positives. The experimental results show that IntRace can effectively detect data race in Racebench, and significantly reduce the number of false positives compared with the existing methods. IntRace had only 5 false positives out of 30 cases, with a defect detection rate of 90.7%. Compared to Rchecker, it reduces the false positive by 73.2%. Manual inspection of all procedures revealed no false negatives.

No. 3 has the most reduction in false positives among all detection programs, from 10 errors to 1. We have analyzed this program and the data race point occurs in two operations $\langle W\#38\# \rangle$, $\langle W\#47 \rangle$. The situation of false positives occurrence point is similar to that in Fig. 3, where the state condition in the main function does not satisfy the path condition for the ISR. The specific operation is shown in Fig. 8. When $i \neq 2$, the operation in line_40 is executed, at which time `svp_simple_007_001_isr_1()` is preempted and the operation on line_47 is executed. However, this operation contradicts the condition in the main function, and the event cannot be executed. IntRace basically eliminates such false positives through access pairs feasibility checking (Sect. 3.3). In No. 8, we eliminated two false positives in Rchecker, the false positives occurred similar to No. 3, both did not consider whether the path is reachable. Such as false positives $\langle R\#50 \rangle$, $\langle W\#68 \rangle$, the condition statement in the task function is `if (svp_simple_004_001_condition4`

Table 3 Benchmark results

No	Size(Loc)	#ISR	#Race	Rchecker				IntRace		
				#Num	#TP	#FP	Time(s)	#Num	#TP	#FP
1	60	3	1	2	1	1	0.36	1	1	0
2	74	2	1	2	1	1	0.51	1	1	0
3	51	1	1	11	1	10	0.37	2	1	1
4	65	2	3	3	3	0	0.30	3	3	0
5	44	2	1	1	1	0	0.28	1	1	0
6	81	1	3	3	3	0	0.35	3	3	0
7	65	1	1	2	1	1	0.36	1	1	0
8	69	2	1	3	1	2	0.31	1	1	0
9	53	1	1	2	1	1	0.32	2	1	1
10	54	3	1	2	1	1	0.31	1	1	0
11	46	2	1	2	1	1	0.47	2	1	1
12	34	1	3	3	3	0	0.36	3	3	0
13	72	1	1	2	1	1	0.37	1	1	0
14	67	3	1	2	1	1	0.45	2	1	1
15	67	1	4	6	4	2	0.34	4	4	0
16	47	1	1	3	1	2	5.20	1	1	0
17	35	1	1	1	1	0	0.31	1	1	0
18	42	1	5	5	5	0	1.81	5	5	0
19	54	1	1	1	1	0	0.32	1	1	0
20	57	3	1	2	1	1	0.35	1	1	0
21	92	1	3	3	3	0	0.31	3	3	0
22	39	1	1	1	1	0	0.31	1	1	0
23	44	1	1	1	1	0	0.29	1	1	0
24	54	1	1	1	1	0	0.41	1	1	0
25	41	1	1	1	1	0	0.31	1	1	0
26	48	1	2	2	2	0	0.34	2	2	0
27	40	1	2	2	2	0	0.30	2	2	0
28	55	2	2	3	2	1	0.32	2	2	0
29	49	3	2	2	2	0	0.32	2	2	0
30	95	1	1	1	1	0	1.40	1	1	0
Sum			49	75		26		54		5

`== 1)`, while interrupt preemption requires the condition statement `if(svp_simple_004_001_condition6≠1)` to be satisfied, so the competition will not happen.

In general, harmful data races can cause reliability and security problems at runtime. We assess the impact of found data race from two aspects based on the theory proposed by Bai et al. [31]. If the shared variable is in the branch conditions such as “if” and “while”, it will affect the control flow, and the related data race is usually considered to be harmful. On the other hand, if shared variables affect the access to

Fig. 8 A point of occurrence of false positives in the No.3 program

```

...      ....
26 void svp_simple_007_001_main() {
...      ....
36 int i = rand();
37 if (i == 2)
38     svp_simple_007_001_global_array[i] = 0x02;
39 else
40     svp_simple_007_001_global_array[i] = 0x09;
42     reader1 = svp_simple_007_001_global_array[2];
43 }
44
45 void svp_simple_007_001_isr_1() {
46     idlerun();
47     svp_simple_007_001_global_array[2] = 0x09;
...      ....
51 }

```

array elements or pointers at runtime, it can largely lead to unexpected behavior of the interrupt program. Among the data races found, we identify that 36 of them to be harmful.

Table 4 presents the data race detection results obtained by IntRace when applied to the real-world interrupt-driven software. Table show the number of remaining races at each stage, and the number of real races that we manually verified. We determine whether reported data races are false positives through developer inspection and existing validation reports. A total of 118 data races were detected in 9 programs, and the number of false positives was 9. We manually check these data races and identify that all of them are real, and no false negatives were found on

Table 4 9 real industrial case results

Name	Pattern matching #Race	PCR analysis		Feasibility checking		Manual checking #Race	#FP
		#Race	Reduction (%)	#Race	Reduction (%)		
mv643xx_eth.c	26	17	34.6	10	41.2	10	0
short	139	63	54.7	20	68.3	18	2
shortprint	6	4	33.3	0	—	0	0
Example1	258	94	63.6	10	89.4	9	1
Example2	326	147	54.9	16	89.1	13	3
Example3	281	162	42.3	11	93.2	9	2
i2c_pca_isa-3	508	213	58.1	24	88.7	24	0
wdt_pci_1	317	151	52.4	25	83.4	24	1
i8xx-tco-1	8	5	37.5	2	60.0	2	0
SUM	1869	856		118		109	9

all programs. We have reported the data races found to the developers of 3 relevant device drivers and have not yet received a reply. 11 harmful data race bugs were detected in the 3 real embedded software (a total of 37 data race bugs were detected), which were confirmed by the validation report. Among the remaining 3 interrupt open source programs, we detected 51 data races, with 1 false positive compared to the previously validated results (50 data race bugs).

Through further examination, we find that there are two main reasons for the false positives. First, there are omission that failed to identify implicit interrupt operations that go through hardware state or related operations to control the program. That means, during the potential concurrency relationship analysis phase, we only summarized common implicit interrupts, while the rest require users to provide configuration files to illustrate this situation. Second, benign data races were not considered, due to the lack of data race intentionally designed for data sharing, although we analyzed path feasibility.

4.3 RQ2: effectiveness of each stage

To better demonstrate the effectiveness of eliminating false positives at each stage of IntRace, we focus on the number of data races remaining after the interleaving pattern matching, potential concurrency relationship analysis, and feasibility analysis stages. As shown in Table 4, columns 2–6 respectively show the number of potential data races at each stage. The evaluation results of each stage consist of two parts, #Race means the number of data race candidates left after the current stage, and “Reduction” is the ratio of reduced races compared to the last stage.

As the results show, in the first stage, an average of 208 potential data races for each project. Potential concurrency relationship analysis reduced the number of false positives contained in matched race candidate pairs by 54.2% overall. The reduction in false positives ranged from 33 to 63% across all projects, leaving 95 potential data races on average for each project to be checked further. The feasibility detection reduced the number of data races reported by potential concurrent analysis by 86.2% overall, with reductions ranging from 41 to 93%. No false negatives were found on all programs by the manual inspection.

4.4 RQ3: efficiency of IntRace

Table 5 show the analysis time for each phase. The evaluation result of each phase consists of two parts, *time(s)* represents the processing time of the current stage, and “percentage” represents the proportion of the total time spent in each stage. Specifically, less time is spent in the matching phase and the potential concurrency analysis phase, with the time spent in the race matching phase accounting for 8.87% of the total testing time. The time spent in potential concurrency analysis accounted for 21.43% of the total testing time. The main time is spent in path feasibility analysis, which accounted for 69.70% of total testing time. IntRace filtered some infeasible access pairs during the first two stages analysis. To ensure correct detection,

Table 5 9 real industrial case times

Name	Pattern matching		PCR analysis		Feasibility checking		Total time (s)
	Time (s)	Percentage (%)	Time (s)	Percentage	Time (s)	Percentage (%)	
mv643xx_eth.c	7.09	12.69	13.97	25.01	34.80	62.03	55.86
short	2.76	12.96	7.89	37.06	10.64	49.98	21.29
shortprint	1.53	5.23	4.13	14.11	23.61	80.66	29.27
Example1	6.78	19.30	9.39	26.73	18.96	53.97	35.13
Example2	5.81	8.01	17.10	23.58	49.62	68.41	72.53
Example3	9.21	10.56	18.56	21.27	59.48	68.17	87.25
i2c_pca_isa-3	2.08	2.40	8.35	9.65	76.13	87.95	86.56
wdt_pci_1	5.13	6.68	20.32	26.44	51.39	66.88	76.84
i8xx-tco-1	3.65	11.47	6.70	21.05	21.48	67.48	31.83

more time was spent on path feasibility checking of candidate pairs. Overall, IntRace took less than 90 s total time on each real industrial program, and can analyze thousands of lines of real industrial software in a relatively short time.

5 Related work

In recent years, a large number of methods on thread-level data race detection have been proposed. However, it can't be ignored that interrupts differ from threads in concurrency structure, control and priority relationships. The unique characteristics of interrupts make existing data race detection techniques inapplicable to interrupt-driven programs. Moreover, most dynamic analysis methods are difficult to easily applied in interrupt-driven programs.

Many previous studies [9, 10, 32] have translated interrupt-driven programs into multi-threaded or serialized programs. But these approaches typically suffer from problems such as high false positives and inefficient execution. Regehr used a source-to-source transformation method to transform interrupt-driven drivers into multi-threaded programs, which were then analyzed using a thread detection tool. However, Regehr [8] transformed the disable-enable instruction to acquire-release locks, ignoring the correctness of the transformation. This cannot prevent interaction with other tasks after interrupt disable, and requires further formal semantics to ensure correctness. Wu et al. [11] proposed a transformation approach that converts interrupt-driven programs into nondeterministic sequential programs based on interrupt semantics, enhancing analysis accuracy by testing path conditions with bounded models. To simulate interrupt semantics, they inserted ISRs indefinitely after each instruction in the main task, causing an exponential increase in program size. Du et al. [33] proposed a program verification enhanced precision analysis method, transforming the vulnerability verification problem into a reachability verification problem on serialized programs, which can detect both data race and atomicity violation defects.

Static analysis is the primary method for detecting data races in interrupt-driven programs. It usually does not produce false negatives but generates a large number of false positives. To address this problem, a series of studies [18, 34–36] have been conducted to improve detection precision. Chopra et al. [19] introduced the concept of the happens-before order and propose disjoint blocks to define the synchronization, through data flow analysis to carry out data race detection. On this basis, Pai et al. [37] extended the use of disjoint blocks to analyze the kernel and check if each conflicting critical access is overwritten by disjoint blocks. However, interrupt-driven programs often use self-organizing synchronization mechanisms, which can result in false positives. Sung et al. [15] proposed an abstract interpretive framework that analyzes each ISR individually as a sequential program to check assertions and propagate results to other ISRs until the end. But they only analyze interruption behavior and do not address the data race problem. Feng et al. [17] proposed Rchecker, an interrupt data race detection tool based on CBMC, which handles synchronization through an interrupt mask list(IML), although scalability issues limit its effectiveness. Li et al. [20] examined atomicity violations in interrupt-driven programs by identifying potential violation candidates through matching access interleaving patterns. They performed modular path pruning by constructing a symbol digest and selecting representative preemption points. But they did not focus on data races. In contrast, IntRace adopts a multi-stage approach to gradually eliminate false positives while effectively detecting data races in industrial interrupt-driven programs.

A few tools [7, 13, 38] use dynamic analysis or hybrid methods to detect data races in interrupt-driven programs. Higashi [39] improved the random scheduling method and set it in the preprocessing stage to trigger interrupts when shared variables are accessed, but it requires substantial manual effort. SimTester [12] utilizes a virtual machine (VM) to identify hardware status through condition triggered interrupts, reducing detection costs. However, the failure to consider interrupt specific event constraints (e.g., priority) leads to large false positives. Wang et al. [14] proposed an automated framework SDRacer for detecting and validating data races, which combines static analysis and symbolic execution to identify potential data races, and check the race statement by dynamic verification through a virtual platform. Nonetheless, simulation is difficult to test all the implementations and cannot guarantee the integrity of the detection (there will be false negatives). Bai et al. [31] proposed a hybrid static-dynamic analysis approach SDILP, to detect data races caused by inconsistent locking discipline in device drivers, but it only detects data races caused by inconsistent locking discipline. At the same time, these methods are difficult to apply in large-scale industrial embedded software due to the substantial overhead of dynamic analysis, which requires building a real or virtual target execution environment.

6 Conclusion

This paper presents IntRace, a static analysis approach for data race detection in interrupt-driven programs. IntRace leverages the interrupt timing, priority, and path feasibility analysis to gradually filter out infeasible competing candidate pairs. We first employ static analysis to efficiently identify potential data race. It then filters out

some access pairs based on potential concurrency analysis. Finally, IntRace deletes the access pairs whose paths are infeasible. The experiments show that our approach can detect data race precisely and efficiently for interrupt-driven programs. In the future, we intend to extend our approach to handle other types of concurrency bugs.

Acknowledgements The authors would like to express appreciation for the financial support provided by the Heilongjiang Natural Science Foundation(JJ2019LH2160).

Author Contributions Jingwen Zhao contributed to Conceptualization of this study, Methodology, Software, Writing—original draft. Yanxia Wu contributed to Supervision, Investigation, Methodology, Writing—review & editing. Jibin Dong contributed to Software, Validation.

Data availability Links to datasets during the current study period are available at <https://github.com/chenruihua/racebench>.

Declarations

Competing interest The authors declare no competing interests.

References

1. Kotker J, Sadigh D, Seshia SA (2011) Timing analysis of interrupt-driven programs under context bounds. In: 2011 Formal methods in computer-aided design (FMCAD), pp 81–90. IEEE
2. Mukherjee S, Kumar A, D'Souza D (2017) Detecting all high-level dataraces in an rtos kernel. In: Verification, Model Checking, and Abstract Interpretation: 18th International Conference, VMCAI 2017, Paris, France, January 15–17, 2017, Proceedings 18, pp 405–423. Springer
3. Fu H, Wang Z, Chen X, Fan X (2018) A systematic survey on automated concurrency bug detection, exposing, avoidance, and fixing techniques. *Softw Qual J* 26:855–889
4. Yu T, Cohen M (2015) Guided test generation for finding worst-case stack usage in embedded systems. In: 2015 IEEE 8th International Conference on Software Testing, Verification and Validation (ICST), pp 1–10. IEEE
5. Poulsen K (2004) Software bug contributed to blackout. *Security Focus*
6. Regehr J (2005) Random testing of interrupt-driven software. In: Proceedings of the 5th ACM International Conference on Embedded Software, pp 290–298
7. Lai Z, Cheung S-C, Chan WK (2008) Inter-context control-flow and data-flow test adequacy criteria for nesc applications. In: Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of Software Engineering, pp 94–104
8. Regehr J, Coopridge N (2007) Interrupt verification via thread verification. *Electron Notes Theor Comput Sci* 174(9):139–150
9. Wu X, Wen Y, Chen L, Dong W, Wang J (2013) Data race detection for interrupt-driven programs via bounded model checking. In: 2013 IEEE Seventh International Conference on Software Security and Reliability Companion, pp 204–210. IEEE
10. Schwarz MD, Seidl H, Vojdani V, Lammich P, Müller-Olm M (2011) Static analysis of interrupt-driven programs synchronized via the priority ceiling protocol. *ACM SIGPLAN Not* 46(1):93–104
11. Wu X, Chen L, Miné A, Dong W, Wang J (2016) Static analysis of runtime errors in interrupt-driven programs via sequentialization. *ACM Trans Embed Comput Syst (TECS)* 15(4):1–26
12. Yu T, Srisa-an W, Rothermel G (2012) Simtester: a controllable and observable testing framework for embedded systems. In: Proceedings of the 8th ACM SIGPLAN/SIGOPS Conference on Virtual Execution Environments, pp 51–62
13. Sun Y, Cheung S-C, Guo S, Cheng M (2019) Disclosing and locating concurrency bugs of interrupt-driven IoT programs. *IEEE Internet Things J* 6(5):8945–8957

14. Wang Y, Gao F, Wang L, Yu T, Zhao J, Li X (2020) Automatic detection, validation, and repair of race conditions in interrupt-driven embedded software. *IEEE Trans Softw Eng* 48(1):346–363
15. Sung C, Kusano M, Wang C (2017) Modular verification of interrupt-driven software. In: 2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE), pp 206–216. IEEE
16. Wang Y, Wang L, Yu T, Zhao J, Li X (2017) Automatic detection and validation of race conditions in interrupt-driven embedded software. In: *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp 113–124
17. Feng H, Yin L, Lin W, Zhao X, Dong W (2020) Rchecker: A cbmc-based data race detector for interrupt-driven programs. In: 2020 IEEE 20th International Conference on Software Quality, Reliability and Security Companion (QRS-C), pp 465–471. IEEE
18. Chen R, Guo X, Duan Y, Gu B, Yang M (2011) Static data race detection for interrupt-driven embedded software. In: 2011 Fifth International Conference on Secure Software Integration and Reliability Improvement-Companion, pp 47–52. IEEE
19. Chopra N, Pai R, D'Souza D (2019) Data races and static analysis for interrupt-driven kernels. In: *Programming Languages and Systems: 28th European Symposium on Programming, ESOP 2019, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2019, Prague, Czech Republic, April 6–11, 2019, Proceedings 28*, pp 697–723. Springer
20. Li C, Chen R, Wang B, Yu T, Gao D, Yang M (2022) Precise and efficient atomicity violation detection for interrupt-driven programs via staged path pruning. In: *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp 506–518
21. Engler D, Ashcraft K (2003) Racexr: effective, static detection of race conditions and deadlocks. *ACM SIGOPS Oper Syst Rev* 37(5):237–252
22. Chen R (2019) Racebench website. <https://github.com/chenruihua/racebench>
23. Praun C (2011) Race detection techniques
24. Huang Y, Zhao Y, Shi J, Zhu H, Qin S (2012) Investigating time properties of interrupt-driven programs. In: *Formal Methods: Foundations and Applications: 15th Brazilian Symposium, SBMF 2012, Natal, Brazil, September 23–28, 2012. Proceedings 15*, pp 131–146. Springer
25. Pratikakis P, Foster JS, Hicks M (2006) Locksmith: context-sensitive correlation analysis for race detection. *Acm Sigplan Not* 41(6):320–331
26. Marek C (2021) DG website. <https://github.com/mchalupa/dg>
27. Shi Q, Xiao X, Wu R, Zhou J, Fan G, Zhang C (2018) Pinpoint: Fast and precise sparse value flow analysis for million lines of code. In: *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pp 693–706
28. Lattner C (2008) Llvm and clang: next generation compiler technology. In: *The BSD Conference*, vol 5, pp 1–20
29. De Moura L, Bjørner N (2008) Z3: an efficient smt solver. In: *Tools and Algorithms for the Construction and Analysis of Systems: 14th International Conference, TACAS 2008, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2008, Budapest, Hungary, March 29–April 6, 2008. Proceedings 14*, pp 337–340. Springer
30. Corbet J, Rubini A, Kroah-Hartman G (2005) *Linux device drivers*. O'Reilly Media, Inc.
31. Bai J-J, Chen Q-L, Jiang Z-M, Lawall J, Hu S-M (2021) Hybrid static-dynamic analysis of data races caused by inconsistent locking discipline in device drivers. *IEEE Trans Softw Eng* 48(12):5120–5135
32. Wu X, Chen L, Miné A, Dong W, Wang J (2015) Numerical static analysis of interrupt-driven programs via sequentialization. In: 2015 International Conference on Embedded Software (EMSOFT), pp 55–64. IEEE
33. Du X, Yin L, Feng H, Dong W (2021) Program verification enhanced precise analysis of interrupt-driven program vulnerabilities. In: 2021 28th Asia-Pacific Software Engineering Conference (APSEC), pp 253–263. IEEE
34. Hsiao C-H, Yu J, Narayanasamy S, Kong Z, Pereira CL, Pokam GA, Chen PM, Flinn J (2014) Race detection for event-driven mobile applications. *ACM SIGPLAN Not.* 49(6):326–336
35. Pan M, Chen S, Pei Y, Zhang T, Li X (2019) Easy modelling and verification of unpredictable and preemptive interrupt-driven systems. In: 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE), pp 212–222. IEEE
36. Schwarz MD, Seidl H, Vojdani V, Apinis K (2014) Precise analysis of value-dependent synchronization in priority scheduled programs. In: *Verification, Model Checking, and Abstract Interpretation*:

- 15th International Conference, VMCAI 2014, San Diego, CA, USA, January 19–21, 2014, Proceedings 15, pp 21–38. Springer
37. Pai R, Singh A, D'Souza D, D'Souza M, Prakash P (2021) Static analysis for detecting high-level races in rtos kernels. *Formal Methods Syst Des*, 1–28
 38. Park S (2013) Fault comprehension for concurrent programs. In: 2013 35th International Conference on Software Engineering (ICSE), pp 1444–1446. IEEE
 39. Higashi M, Yamamoto T, Hayase Y, Ishio T, Inoue K (2010) An effective method to control interrupt handler for data race detection. In: *Proceedings of the 5th Workshop on Automation of Software Test*, pp 79–86

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.